Final Exam – DATA 3401 (Spring 2026)

Start Date: 04/27

Due Date: 5/1 (at 11:59pm) – this is a hard deadline so don't miss it!

Final Rules

Please work the exercises below **on your own**. When you have completed the exam, you should push your completed jupyter notebook to your CANVAS for this class in the **Assignments->Final Exam** folder.

You may not discuss the problems with **anyone else**, including persons on an online internet forum. Consulting an outside source like this will be considered an academic integrity violation. **Any questions should be referred to Dr. Ce Bian.**

You may use all class resources including previous labs and lectures, and anything posted on the course CANVAS.

You may not use any function that trivializes a problem. For example, if I ask you to write a `max` function that computes the maximum entry in a list, you are not allowed to use the pre-defined Python function `max`; you must write your own.

Each problem below (including the Bonus) will be worth approximately 20 points. Achieving over 100 on this exam is possible if the Bonus is completed.

Exercise 1

Download the adult.names and adult.data files from the Final folder in the course GitHub repo.

1. Print the adult.names file in your jupyter notebook
2. Using pandas, make a dataframe from the adult.data file and print its `head()`
3. Delete the column titled 'fnlwgt' from your dataframe
4. Plot a histogram showing how many people from each working class make above 50k per year and below 50k per year
5. Similarly, plot a histogram showing how many people from each education level make above and below 50k per year
6. Choose one other category and plot a histogram similar to the above
7. Discuss the results of these histograms. What (if any) conclusions can you draw from them?

In [103…

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

In [104…

```
with open('adult.names', 'r') as f:
    print(f.read())

# Read csv, print the head. Using display otherwise it doensnt show up for some reason.
df = pd.read_csv('adult.data')
display(df.head())

# Dropping the relevant column.
df = df.drop(columns=['fnlwgt'])
```

| This data was extracted from the census bureau database found at
| http://www.census.gov/ftp/pub/DES/www/welcome.html
| Donor: Ronny Kohavi and Barry Becker,
| Data Mining and Visualization
| Silicon Graphics.
| e-mail: ronnyk@sgi.com for questions.
| Split into train-test using MLC++ GenCVFiles (2/3, 1/3 random).
| 48842 instances, mix of continuous and discrete   (train=32561, test=16281)
| 45222 if instances with unknown values are removed (train=30162, test=15060)
| Duplicate or conflicting instances : 6
| Class probabilities for adult.all file
| Probability for the label '>50K' : 23.93% / 24.78% (without unknowns)
| Probability for the label '<=50K' : 76.07% / 75.22% (without unknowns)
|
| Extraction was done by Barry Becker from the 1994 Census database. A set of
| reasonably clean records was extracted using the following conditions:
| ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HRSWK>0))
|
| Prediction task is to determine whether a person makes over 50K
| a year.
|
| First cited in:
| @inproceedings{kohavi-nbtree,
| author={Ron Kohavi},
| title={Scaling Up the Accuracy of Naive-Bayes Classifiers: a
| Decision-Tree Hybrid},
| booktitle={Proceedings of the Second International Conference on
| Knowledge Discovery and Data Mining},
| year = 1996,
| pages={to appear}}
|
| Error Accuracy reported as follows, after removal of unknowns from
| train/test sets):
| C4.5   : 84.46+-0.30
| Naive-Bayes: 83.88+-0.30
| NBTree   : 85.90+-0.28
|
|
| Following algorithms were later run with the following error rates,
| all after removal of unknowns and using the original train/test split.
| All these numbers are straight runs using MLC++ with default values.
|
| Algorithm   Error
| -- ----------------   -----
| 1 C4.5   15.54
| 2 C4.5-auto   14.46
| 3 C4.5 rules   14.94
| 4 Voted ID3 (0.6)   15.64
| 5 Voted ID3 (0.8)   16.47
| 6 T2   16.84
| 7 1R   19.54
| 8 NBTree   14.10
| 9 CN2   16.00
| 10 HOODG   14.82
| 11 FSS Naive Bayes   14.05
| 12 IDTM (Decision table)   14.46
| 13 Naive-Bayes   16.12
| 14 Nearest-neighbor (1)   21.42
| 15 Nearest-neighbor (3)   20.35
| 16 OC1   15.04
| 17 Pebls   Crashed. Unknown why (bounds WERE increased)
|

| Conversion of original data as follows:
| 1. Discretized agrossincome into two ranges with threshold 50,000.
| 2. Convert U.S. to US to avoid periods.
| 3. Convert Unknown to "?"
| 4. Run MLC++ GenCVFiles to generate data,test.
|
| Description of fnlwgt (final weight)
|
| The weights on the CPS files are controlled to independent estimates of the
| civilian noninstitutional population of the US. These are prepared monthly
| for us by Population Division here at the Census Bureau. We use 3 sets of
| controls.
| These are:
| 1. A single cell estimate of the population 16+ for each state.
| 2. Controls for Hispanic Origin by age and sex.
| 3. Controls by Race, age and sex.
|
| We use all three sets of controls in our weighting program and "rake" through
| them 6 times so that by the end we come back to all the controls we used.
|
| The term estimate refers to population totals derived from CPS by creating
| "weighted tallies" of any specified socio-economic characteristics of the
| population.
|
| People with similar demographic characteristics should have
| similar weights. There is one important caveat to remember
| about this statement. That is that since the CPS sample is
| actually a collection of 51 state samples, each with its own
| probability of selection, the statement only applies within
| state.

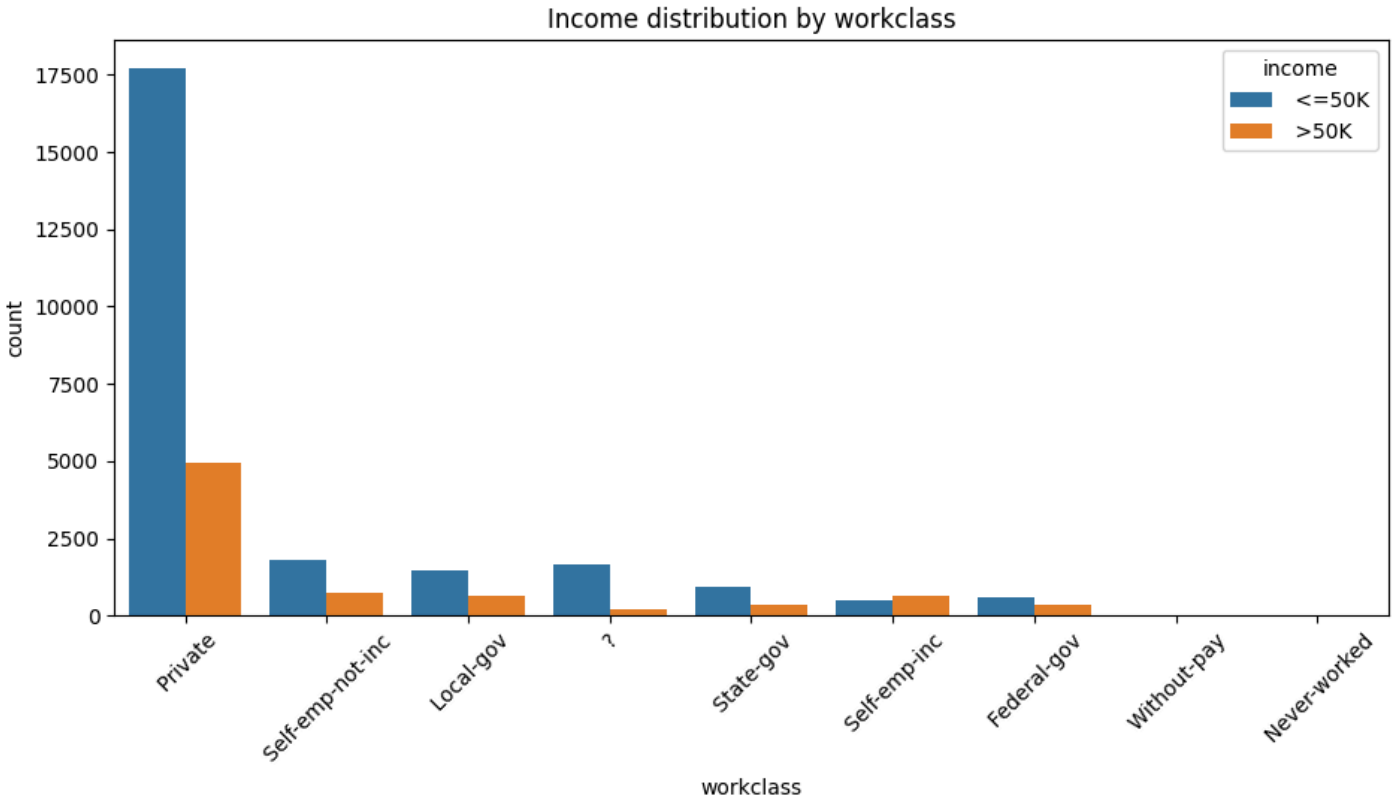
>50K, <=50K.

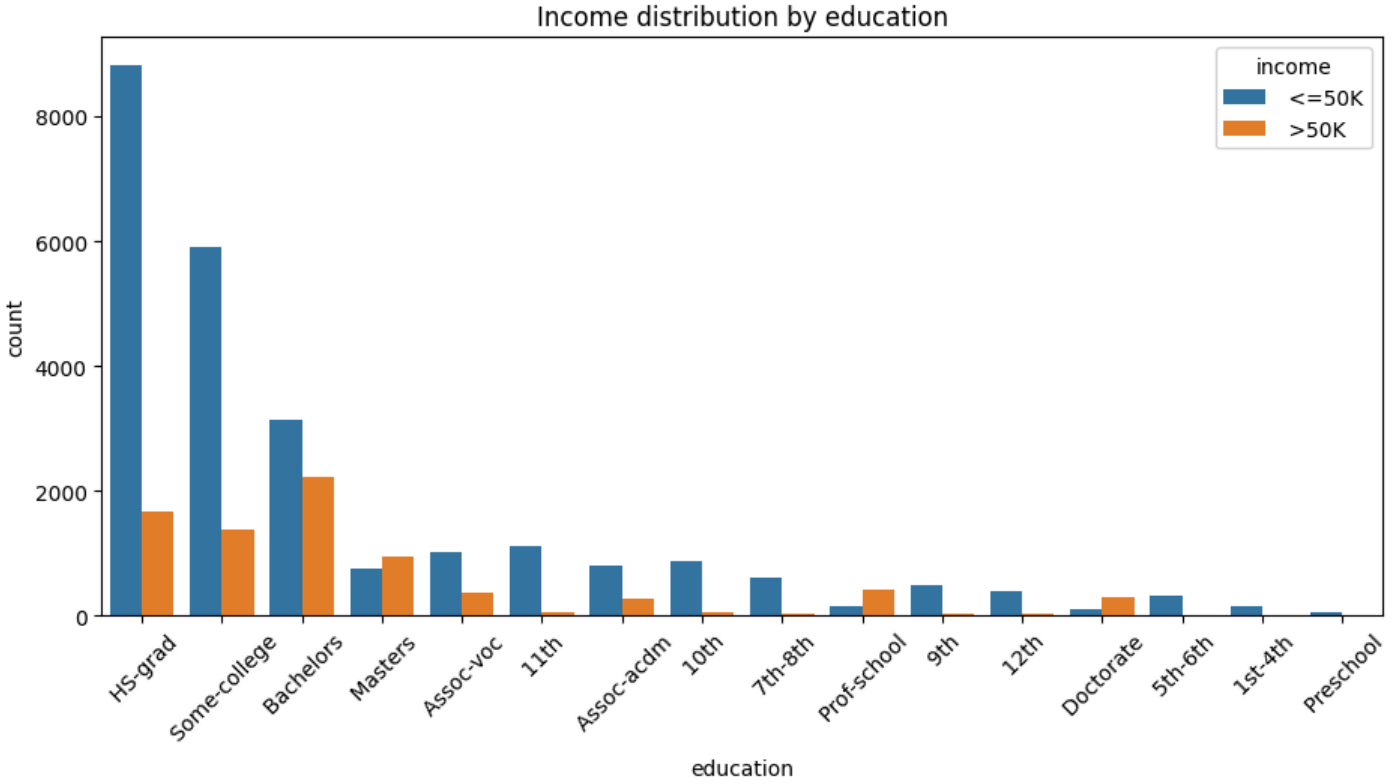
age: continuous.
workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pa
y, Never-worked.
fnlwgt: continuous.
education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8
th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
education-num: continuous.
marital-status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-a
bsent, Married-AF-spouse.
occupation: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Ha
ndlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-s
erv, Protective-serv, Armed-Forces.
relationship: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex: Female, Male.
capital-gain: continuous.
capital-loss: continuous.
hours-per-week: continuous.
native-country: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-
USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland,
Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, H
aiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trina
dad&Tobago, Peru, Hong, Holand-Netherlands.

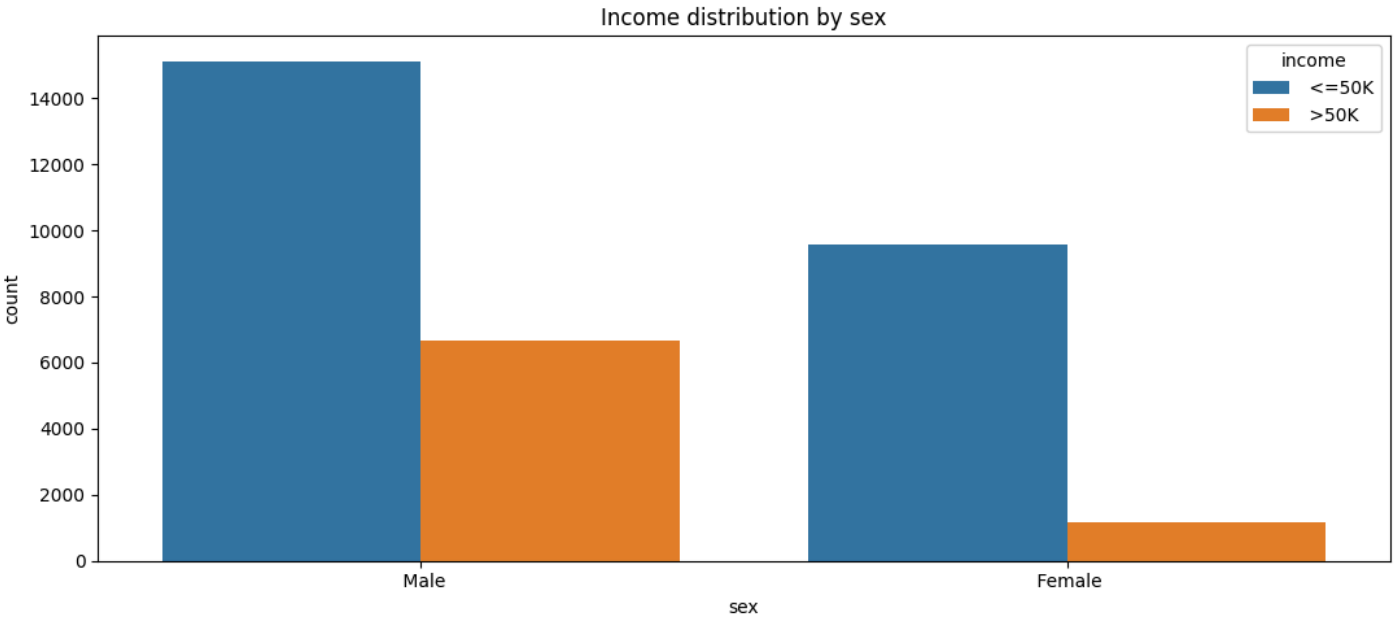
	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capita ga
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	21
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	

In [104…

```
plt.figure(figsize=(11, 5))
order = df['workclass'].value_counts().index
sns.countplot(data=df, x='workclass', hue='income', order=order)
plt.title('Income distribution by workclass')
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(11, 5))
order = df['education'].value_counts().index
sns.countplot(data=df, x='education', hue='income', order=order)
plt.title('Income distribution by education')
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(11, 5))
order = df['sex'].value_counts().index
sns.countplot(data=df, x='sex', hue='income', order=order)
plt.title('Income distribution by sex')
plt.tight_layout()
plt.show()

print('''
Discussion:
Income by workclass:
It's quite clear the most common income source is from private industry, the next being
self employed. I found this surprising, I thought there would have been far many more
government workers than self employed workers.
Income by education:
This plot shows the ratio between workers making >50k and < 50k starts to level out based
on your education level. HS Grads have a huge portion making less than 50k, whereas masters
grads have more people earning more than 50k than those who make less. Funny enough, before
attending college, I only had a 7th grade education level.
Icome by sex:
This has been debated to the end of the earth. The quantity of women earning
> 50k is a fraction of the quantity of men making >50k. The overall population is also much
higher among men.
''')
```


Income distribution by workclass
income
<=50K
>50K
count
17500
15000
12500
10000
7500
5000
2500
0
Private
Self-emp-not-inc
Local-gov
?
State-gov
Self-emp-inc
Federal-gov
Without-pay
Never-worked
workclass


Income distribution by education
income
<=50K
>50K
count
8000
6000
4000
2000
0
HS-grad
Some-college
Bachelors
Masters
Assoc-voc
11th
Assoc-acdm
10th
7th-8th
Prof-school
9th
12th
Doctorate
5th-6th
1st-4th
Preschool
education




```
Discussion:
Income by workclass:
It's quite clear the most common income source is from private industry, the next being
self employed. I found this surprising, I thought there would have been far many more
government workers than self employed workers.
Income by education:
This plot shows the ratio between workers making >50k and < 50k starts to level out based
on your education level. HS Grads have a huge portion making less than 50k, whereas masters
grads have more people earning more than 50k than those who make less. Funny enough, before
attending college, I only had a 7th grade education level.
Icome by sex:
This has been debated to the end of the earth. The quantity of women earning
> 50k is a fraction of the quantity of men making >50k. The overall population is also much
higher among men.
```

Exercise 2

If you invest a principal value of P at time 0, and interest is continuously compounded at a rate r ($0 < r < 1$), then the amount of money you would have after t years is

$$M(t) = Pe^{rt}.$$

Fry has \$0.01 in his bank account, but is accidentally transported 1,000 years into the future. He returns to his bank (which luckily still exists) to see how much money he now has. Assume Fry's account earns continuously compounded interest at a rate of 5% (or $r = 0.05$).

1. Create a numpy array of time from 0 to 1,000 years increasing by 1 year
2. Create a new numpy array that calculates how much money Fry's account has at each year
3. Plot Fry's money over the given timeframe

Now if Fry only earned simple interest, the amount of money he would have after t years would be

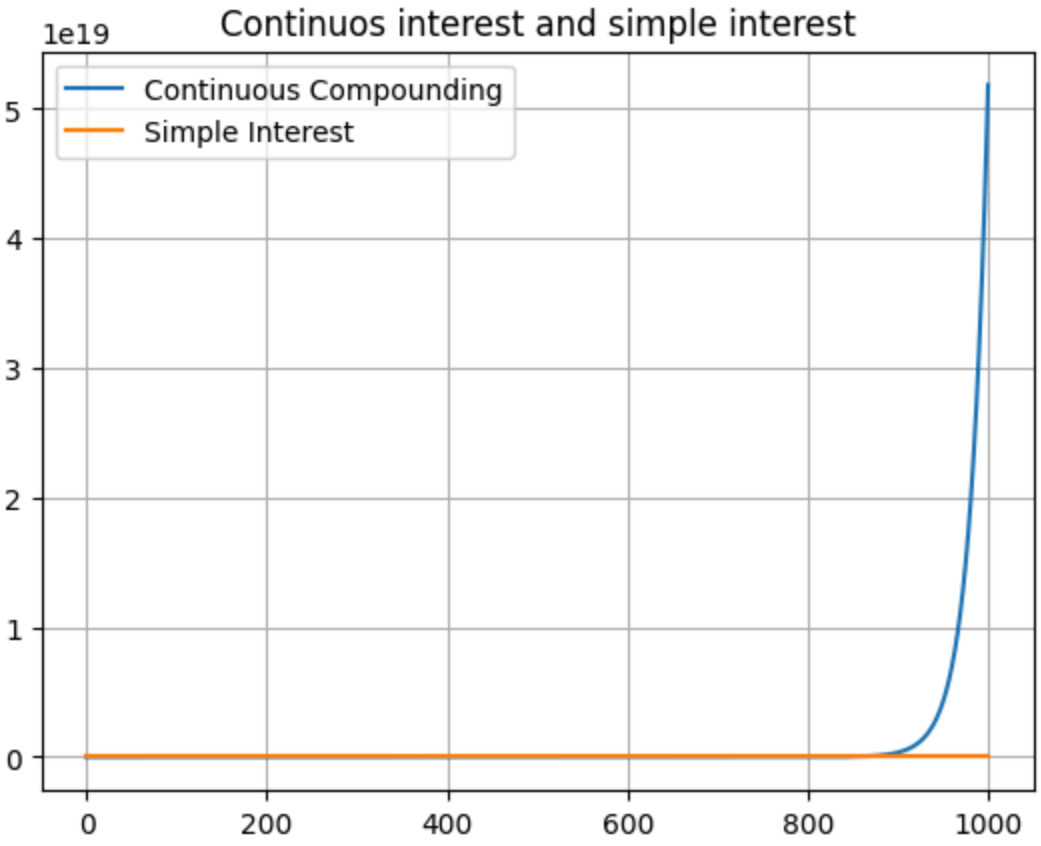
$$S(t) = P(1 + rt).$$

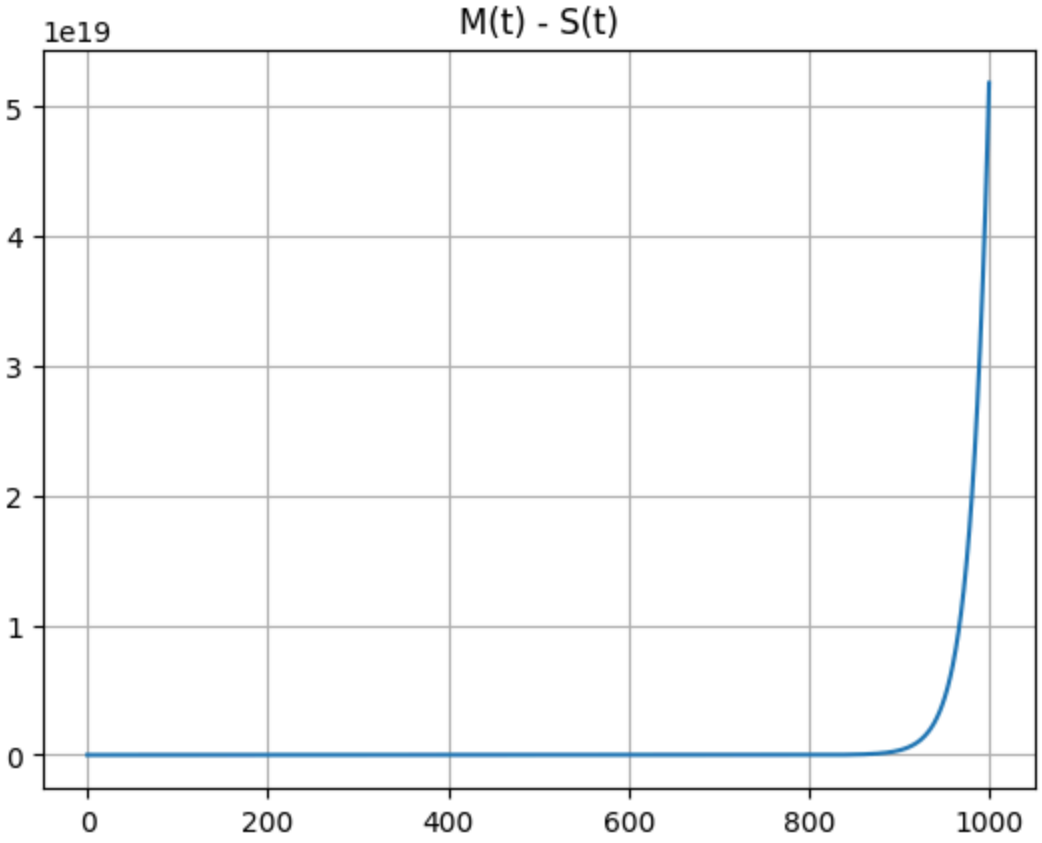
1. Perform the same procedure as steps 1-3 above assuming Fry earns 5% simple interest.
2. To illustrate the difference, also plot $M(t) - S(t)$ over the timeframe.
3. What do you conclude?

Aside: This problem, while silly, should teach you an important lesson: invest your money as early as you possibly can!

In [104…

```
initial_balance = 0.01
interest_rate = 0.05

timeframe = np.arange(0,1001) # This is an array starting at year zero, going up to year 1000, 1
m_t = initial_balance * np.power(np.e, interest_rate * timeframe) # The continuous interest form

s_t = initial_balance * (1 + interest_rate * timeframe)
plt.plot(timeframe, m_t, label='Continuous Compounding')
plt.plot(timeframe, s_t, label='Simple Interest')
plt.title('Continuos interest and simple interest')
plt.legend()
plt.grid()
plt.show()

plt.plot(timeframe, m_t - s_t)
plt.title('M(t) - S(t)')
plt.grid()
plt.show()
print('Conclusion: \n' \
'Continuous interest outweighs simple interest by so much \n' \
'that it\'s not even visible on the same graph.')
```






```
Conclusion:
Continuous interest outweighs simple interest by so much
that it's not even visible on the same graph.
```

Exercise 3

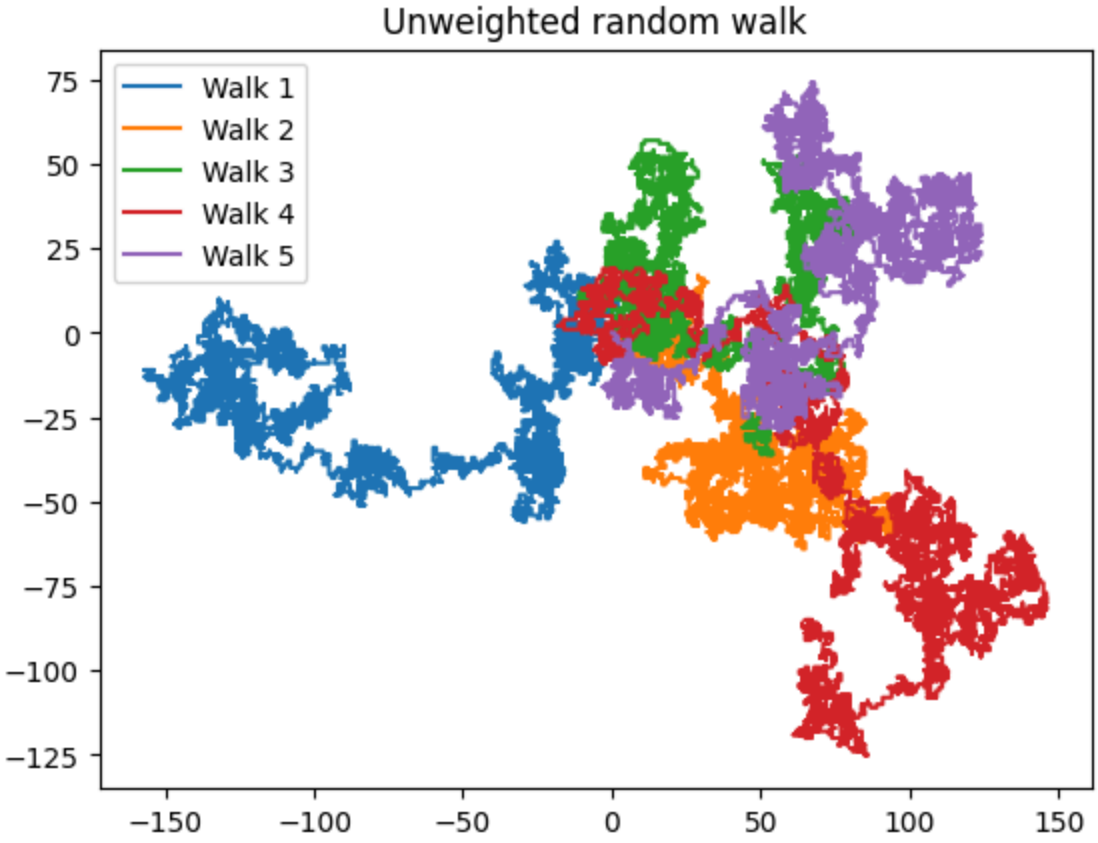
In Lab 7 you coded a random walk in 1-dimension. Here you will code a 2-dimensional random walk.

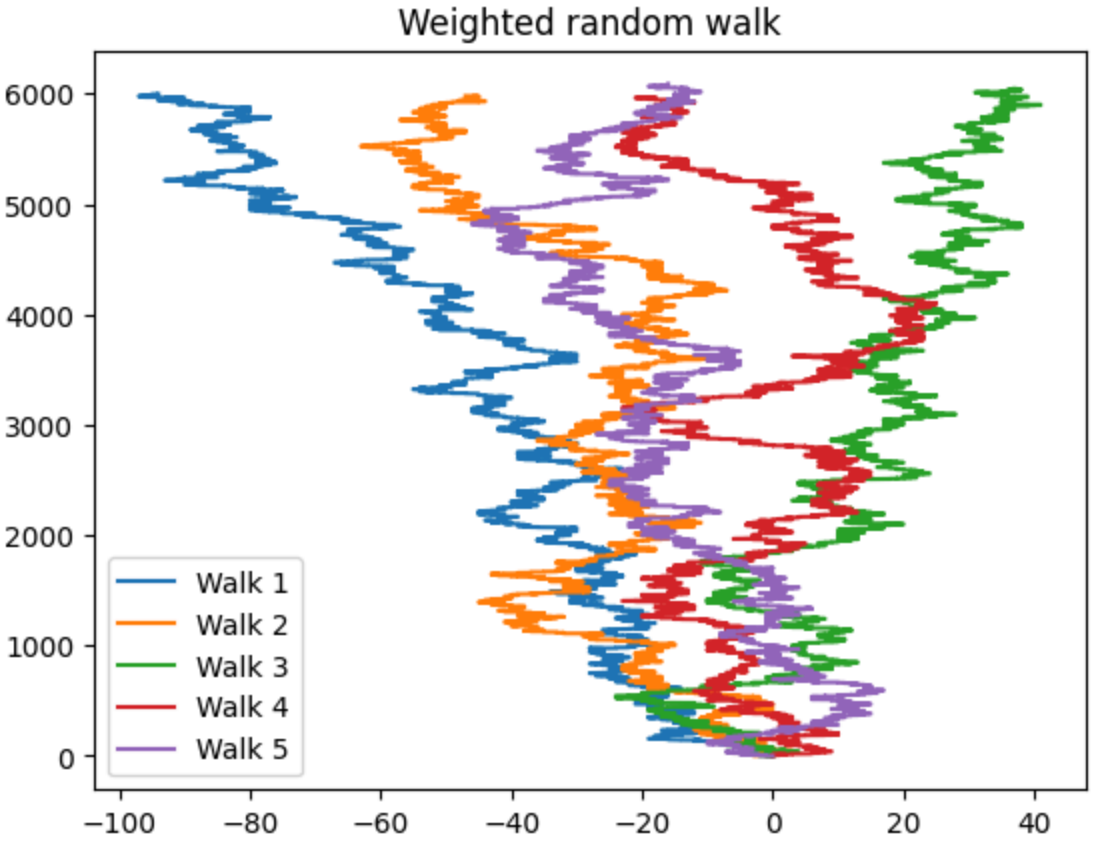
1. As before, write a function `random_walk(num_steps,init_position)` that takes an integer for `num_steps` corresponding to how many steps the random walker will take, and a numpy array for `init_position` with x and y coordinates of the starting point. The function should return the full sequence of positions of the walker as they take `num_steps` steps
2. To determine the next location of the walker at a given step, the walker uniformly randomly chooses to walk one of 4 possible directions: up, down, left, or right and takes a step of length 1 in either direction (e.g., if the walker starts at (0,0), their possible moves for the first step are up=(0,1), down=(0,-1), left=(-1,0), right=(1,0))
3. Call your `random_walk` function 5 times with initial position of (0,0) and 10,000 steps and plot the random walk for each function call using pyplot (here is an example of what it should look like; if the image does not render in GitHub, open 2drandomwalk.png in the Final folder) 2drandomwalk.png
4. What do you notice?
5. Modify your random walk such that the walker is more likely to travel up (specifically, they will step up with probability .7 and the remaining directions each with probability .1).
6. Call your modified random walk function 5 times and plot the results.
7. What do you notice? Is there a difference in behavior compared to the original version?

In [104…

```python
def random_walk(num_steps:int, init_position:np.ndarray):
    step_sequence = [init_position.copy()] # Fix for starting position at [0,0]
    for i in range(1, num_steps + 1):
        if np.random.choice(['x', 'y']) == 'x':
```

```
            # Changing the x position.
            init_position[0] += np.random.choice([-1,1])
        else:
            # Changing the y position.
            init_position[1] += np.random.choice([-1,1])
        step_sequence.append(init_position.copy())
    return step_sequence # Sequence of steps

# I stacked the plots to save space.
for m in range(5):
    walk = random_walk(10000, np.array([0, 0]))
    walk_x = [i[0] for i in walk]
    walk_y = [i[1] for i in walk]
    plt.plot(walk_x, walk_y)
plt.title('Unweighted random walk')
plt.legend([f'Walk {i + 1}' for i in range(5)])
plt.show()

# Complete rewrite because I did not account for weights at all in the first one.
def random_walk(num_steps:int, init_position:np.ndarray):
    step_sequence = [init_position.copy()]

    direction_weighted = {'up':0.7,
                          'down':0.1,
                          'left':0.1,
                          'right':0.1}

    for i in range(1, num_steps + 1):
        direction = np.random.choice(list(direction_weighted.keys()),
                                     p=list(direction_weighted.values()))

        if direction == 'up':
            init_position[1] += 1
        elif direction == 'down':
            init_position[1] -= 1
        elif direction == 'left':
            init_position[0] -= 1
        elif direction == 'right':
            init_position[0] += 1

        step_sequence.append(init_position.copy())
    return step_sequence


for plot in range(5):
    walk = random_walk(10000, np.array([0, 0]))

    walk_x = [i[0] for i in walk]
    walk_y = [i[1] for i in walk]

    plt.plot(walk_x,walk_y)

plt.title('Weighted random walk')
plt.legend([f'Walk {i + 1}' for i in range(5)])
plt.show()

print('Obersvation:\n' \
'The unweighted random walk tends to make shapes around random clusters of positions.\n' \
'In the weighted version, the plot displays the tendency to choose the +y axis. \n' \
'The range of values between them is very different, the weighted random walk reaches \n' \
'up to position 6000+ on the y axis whereas the unweighted random walk will usually stay below 2
```





Obersvation:
The unweighted random walk tends to make shapes around random clusters of positions.
In the weighted version, the plot displays the tendency to choose the +y axis.
The range of values between them is very different, the weighted random walk reaches
up to position 6000+ on the y axis whereas the unweighted random walk will usually stay below 200
and all axis.

Exercise 4

Download the Shakespeare.txt file for this problem.

1. Print the file in your notebook. **Steps 1 and 2 below can be done in whichever order you like**
2. Take out all line-break characters from the file
3. Delete all instance of "the", "in", "and"
4. Print the modified file

In [104…

```
# Reading the file and printing it.
print('The original file:')
with open('Shakespeare.txt') as f:
    print(f.read())
```

```
The original file:
So is it not with me as with that Muse
Stirr'd by a painted beauty to his verse,
Who heaven itself for ornament doth use
And every fair with his fair doth rehearse
Making a couplement of proud compare,
With sun and moon, with earth and sea's rich gems,
With April's first-born flowers, and all things rare
That heaven's air in this huge rondure hems.
O' let me, true in love, but truly write,
And then believe me, my love is as fair
As any mother's child, though not so bright
As those gold candles fix'd in heaven's air:
Let them say more than like of hearsay well;
I will not praise that purpose not to sell.
```

In [104…

```
# New code cell because the printout was being truncated due to length.
print('\nThe new file:')
new_text = []
with open('Shakespeare.txt') as f:
    for i in f:
        new_text.append(i.replace('\n', '')
                        .replace('the','')
                        .replace('in','')
                        .replace('and','')
                        .replace('And',''))

# I went with printing it line-by-line since printing all on one line was not explicitly mentione
# Removing line break characters sort of implies that it should be on one line, but not 100% sure
for line in new_text:
    print(line)
```

```
The new file:
So is it not with me as with that Muse
Stirr'd by a pated beauty to his verse,
Who heaven itself for ornament doth use
 every fair with his fair doth rehearse
Makg a couplement of proud compare,
With sun  moon, with earth  sea's rich gems,
With April's first-born flowers,  all thgs rare
That heaven's air  this huge rondure hems.
O' let me, true  love, but truly write,
 n believe me, my love is as fair
As any mor's child, though not so bright
As those gold cles fix'd  heaven's air:
Let m say more than like of hearsay well;
I will not praise that purpose not to sell.
```

Exercise 5

1. Create a `card` class that has two attributes: `value` and `suit`. A card should be initialized by passing it both the value and suit.
2. Create a `deck` class to represent a deck of cards.
    A. Each card in the deck should be of type card (the class you made in Step 1)
    B. Your deck should be a list of 52 cards (see midterm for card description if needed)
    C. Your deck should be ordered, and should have the option of `shuffled` (True or False) upon initialization. If `shuffled` is False, the deck should be in order (Ace-2 of Spades, Ace-2 of Clubs, Ace-2 of Diamonds, Ace-2 of Hearts), otherwise if `shuffled` is True, the deck should be randomly ordered
    D. Create a method `deal_cards(num_cards)` that chooses the **first** `num_cards` cards from the deck and returns a list of these cards. This method should also remove the cards from the deck.
    E. Create a `shuffle` method to shuffle the deck.
3. Create a subclass of `deck` called `hand`
    A. A `hand` will be a list of cards
    B. A `hand` should be initialized with an option of `num_cards` for the number of cards in the hand, and should be initialized via the `deal_cards` method of your `deck` class
    C. Create (or repurpose your old function) a method to check if the hand is a flush, and if so determine the suit
    D. Create a method to check if a hand is a straight (consecutive cards; for simplicity, assume Ace is high (i.e., Ace, King, ..., 2 is the order of the numbers))
4. Using your classes, create an unshuffled deck, and deal 5 hands of 5 cards each.
    A. Check if each hand is a flush, and report the results along with the suits of the flush(es).
    B. Check if each hand is a straight and report the results
5. Create a shuffled deck and deal 9 hands of 5 cards. Check for straights and flushes and report the results.

In [104…

```python
suit_list = ['Spades', 'Clubs', 'Diamonds', 'Hearts']
value_list = ['Ace', 2, 3, 4, 5, 6, 7, 8, 9, 10, 'Jack', 'Queen', 'King']

import random
class card:
    def __init__(self, value, suit):
        self.value = value
        self.suit = suit
```

```
    def __repr__(self):
        return f"{self.value} of {self.suit}"

class deck:
    def __init__(self, shuffled=False):
        # The list of cards, with elements being instances of the card class.
        self.shuffled = shuffled
        self.cards = [card(value, suit) for suit in suit_list for value in value_list]
        if shuffled:
            random.shuffle(self.cards)

    def deal_cards(self, num_cards):
        # Deal up to the number of cards, cards left are only after the
        dealt = self.cards[:num_cards]
        self.cards = self.cards[num_cards:]
        return dealt

    def shuffle(self):
        random.shuffle(self.cards)

    def __repr__(self):
        return str(self.cards)

class hand(deck):
    def __init__(self, source_deck, num_cards):
        self.cards = source_deck.deal_cards(num_cards)
        self.shuffled = False

    def check_flush(self):
        suit_set = set()
        for card in self.cards:
            suit_set.add(card.suit)

        if len(suit_set) == 1:
            return True, str(suit_set.pop())
        else:
            return False, None

    def check_straight(self):
        face_card_values = {'Jack': 11, 'Queen': 12, 'King': 13, 'Ace': 14}
        numeric_values = [
            face_card_values[c.value] if c.value in face_card_values else c.value
            for c in self.cards
        ]
        low = min(numeric_values)
        n = len(numeric_values)
        expected_sum = sum(range(low, low + n))
        actual_sum = sum(numeric_values)
        no_duplicates = len(numeric_values) == len(set(numeric_values))

        return expected_sum == actual_sum and no_duplicates
# The unshuffled deck.
print('Unshuffled deck:')
deck_1 = deck(shuffled=False)
for i in range(5):
    hand_1 = hand(deck_1, 5)
    is_flush, suit = hand_1.check_flush()
    is_straight = hand_1.check_straight()
    print(f'Is flush: {is_flush}, Suit: {suit}, is straight: {is_straight}')


print('\nThe shuffled deck:')
```

```
deck_2 = deck(shuffled=True)
for i in range(9):
    hand_2 = hand(deck_2, 5)
    is_flush, suit = hand_2.check_flush()
    is_straight = hand_2.check_straight()
    print(f'Is flush: {is_flush} Suit: {suit}, is straight: {is_straight}')


# This while loop shows how many hands it takes to get a straight.
hand_count = 0
while True:
    my_deck = deck(shuffled=True)

    my_hand = hand(my_deck, 5)
    hand_count += 1
    if my_hand.check_straight() != False:
        print(f'It took {hand_count} hands to get a straight!')
        break
```

```
Unshuffled deck:
Is flush: True, Suit: Spades, is straight: False
Is flush: True, Suit: Spades, is straight: True
Is flush: False, Suit: None, is straight: False
Is flush: True, Suit: Clubs, is straight: True
Is flush: True, Suit: Clubs, is straight: True

The shuffled deck:
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
Is flush: False Suit: None, is straight: False
It took 366 hands to get a straight!
```

Bonus (20 points)

Take part in the Spaceship Titanic Challenge on Kaggle [https://www.kaggle.com/competitions/spaceship-titanic/overview]

To receive points on this question, you must make a **full attempt** at a solution to the competition. You **must** put your code in your notebook, and **must** include a screenshot of the Kaggle submission notification. Grading for this problem will be based on correctness and complexity of a solution, not solely on attempt.

In [104…

```
# Write your code here
```